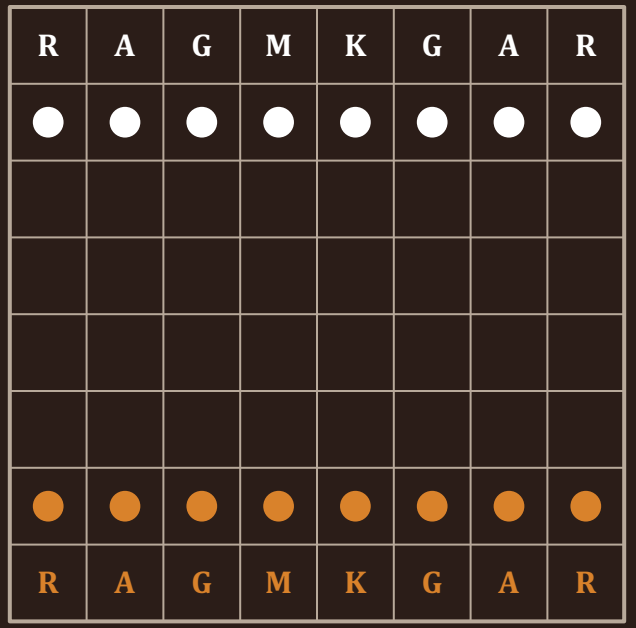
ChaturangZero

Learning Chaturanga by self-play, AlphaZero style

CS F407 course project



Chaturanga: the ancestor of chess

Piece	How it moves
Raja (king)	One square in any direction
Mantri (minister)	One square diagonally
Gaja (elephant)	Jumps exactly two squares diagonally
Ashva (horse)	Like a knight
Ratha (chariot)	Like a rook
Padati (soldier)	One step forward, no double step; becomes a Mantri on the last rank

Why it is hard to learn

- Weak, short-range pieces make attacks slow
- Checkmate = win, stalemate = draw
- Draws by 100 moves without progress or threefold repetition
- So most games between beginners end in a draw

The course rules we worked within

0

domain knowledge

Only the rules of the game. No piece values, no opening books, no human games

50

simulations per move

In training (the limit is 30 to 50). Play time may use more

4×64

residual network

4 blocks of 64 filters (limit 4 to 6 blocks, 64 to 128 filters), 836k parameters

4096

possible moves

Every from-square × to-square pair, illegal moves masked before the softmax

How it learns: the AlphaZero loop

1 Self-play

The agent plays itself. Each move: 50 steps of tree search guided by the network

2 Replay buffer

Every position is stored with the search's move preferences and the final result

3 Training

The network learns to predict both: policy (cross-entropy) + value (MSE) + L2

4 New network

The improved network plays the next round of games

Search: PUCT selection, the network's move priors to expand, its value to judge leaves (no random playouts). **Exploration:** Dirichlet noise at the root in self-play.

What we built

1. **Rules engine**
   Student 1
   Board, legal moves, check, game end; 13-plane 8×8 encoding

2. **Network**
   Student 2
   Residual net, policy + value heads, training loop, replay buffer

3. **Search + evaluation**
   Student 3
   MCTS with PUCT, benchmarks, matches, human-game logging, plots

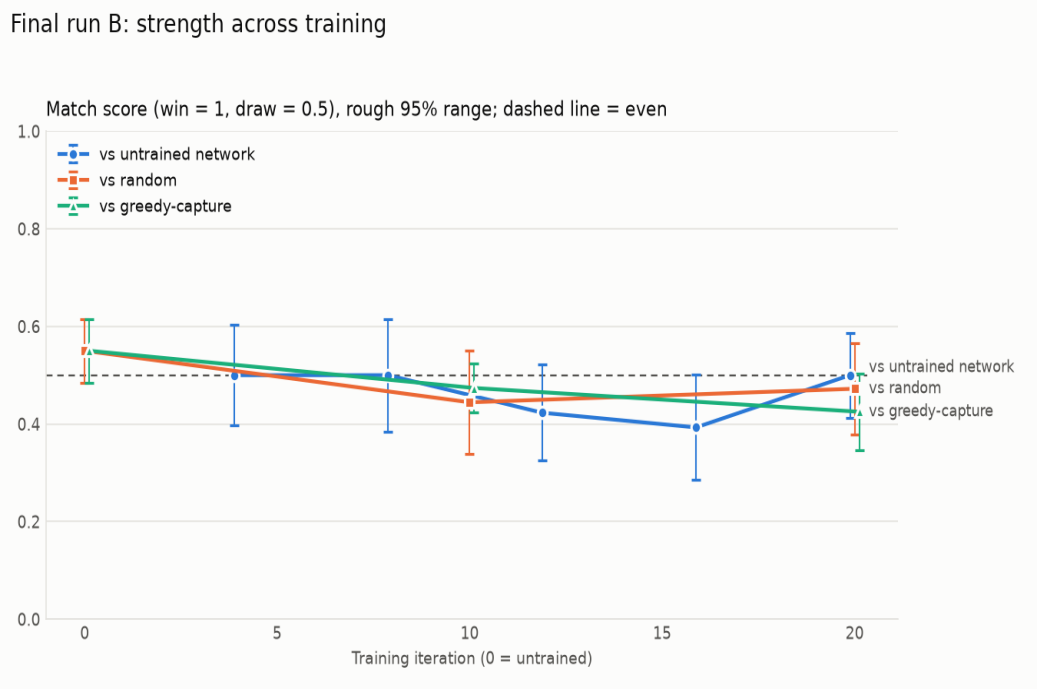
4. **Browser game**
   Shared
   Play the final model with clickable moves and a view of what it thinks

164

automated tests

36 for the rules alone; the fast engine is checked against the original on thousands of positions

First runs: no measurable improvement



400 games per run

- Scores stayed near 0.5 against random, greedy-capture and untrained players
- About 80% of self-play games were draws
- About 7 hours per run on one CPU core
- A team member new to chess beat it: 3 wins, 3 draws

Why, and what we changed

Diagnosis

- Draws give the value head a target of 0: almost nothing to learn
- 400 games is tiny: AlphaZero used about 44 million
- So the problem was data, and data was limited by speed

259 → 19 ms

per move: faster rules engine and a folded network

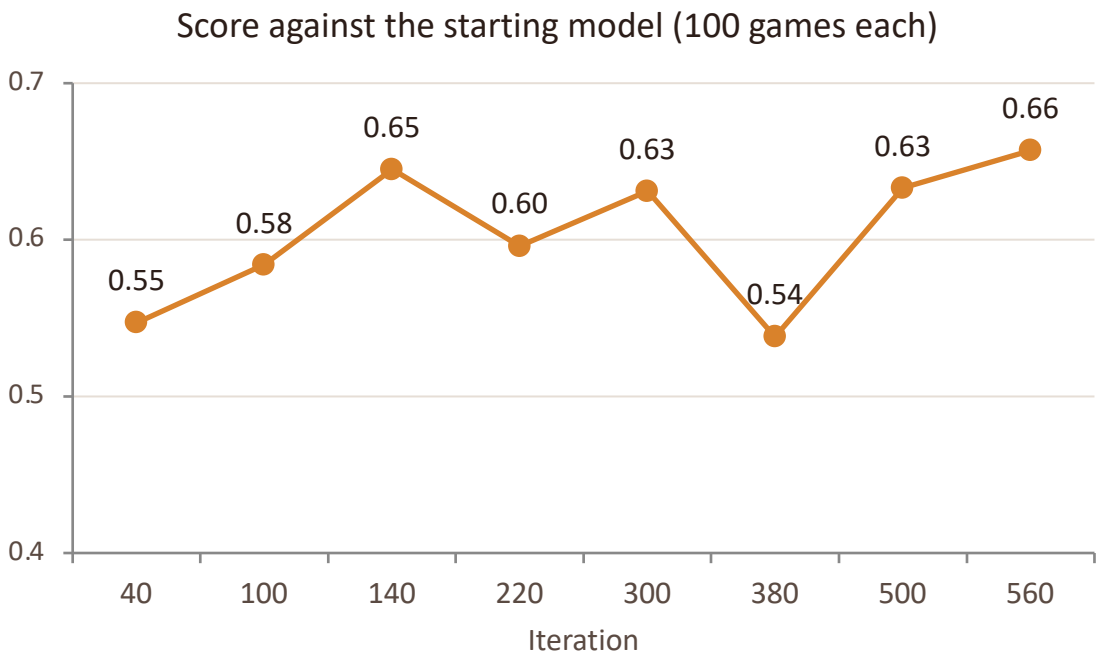
6

games in parallel, one per core

2.2–2.4×

faster again with self-play on the laptop GPU

The long run: training finally helps



54,000

self-play games

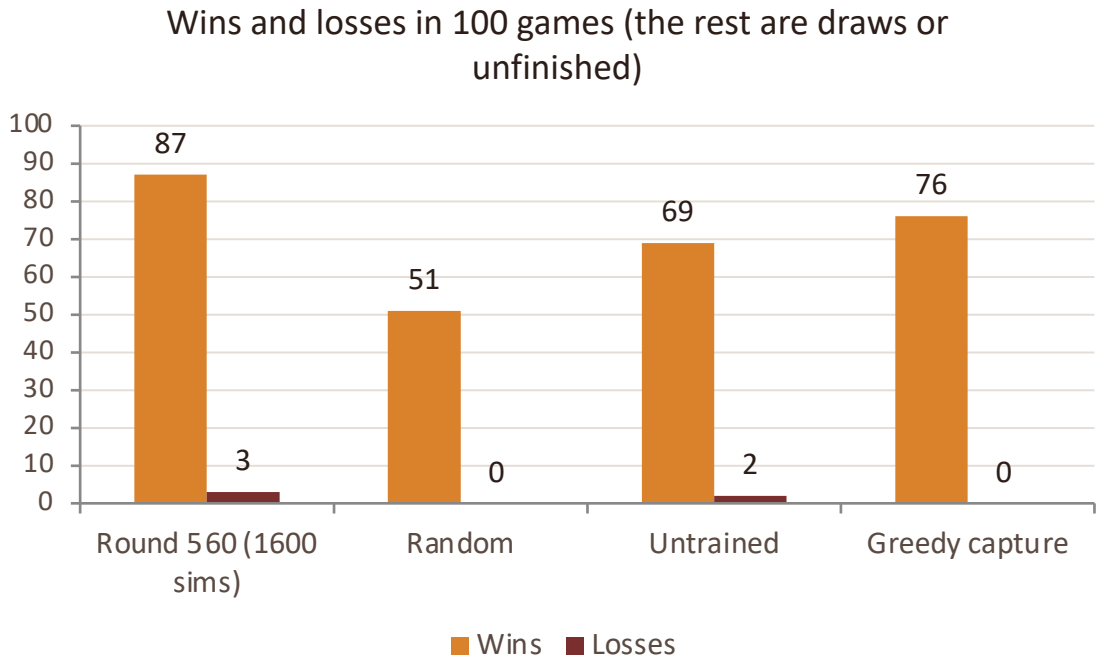
144 h

on one laptop

27–0

round 560 vs its starting point (59 draws)

Final model (round 1280) vs fixed opponents



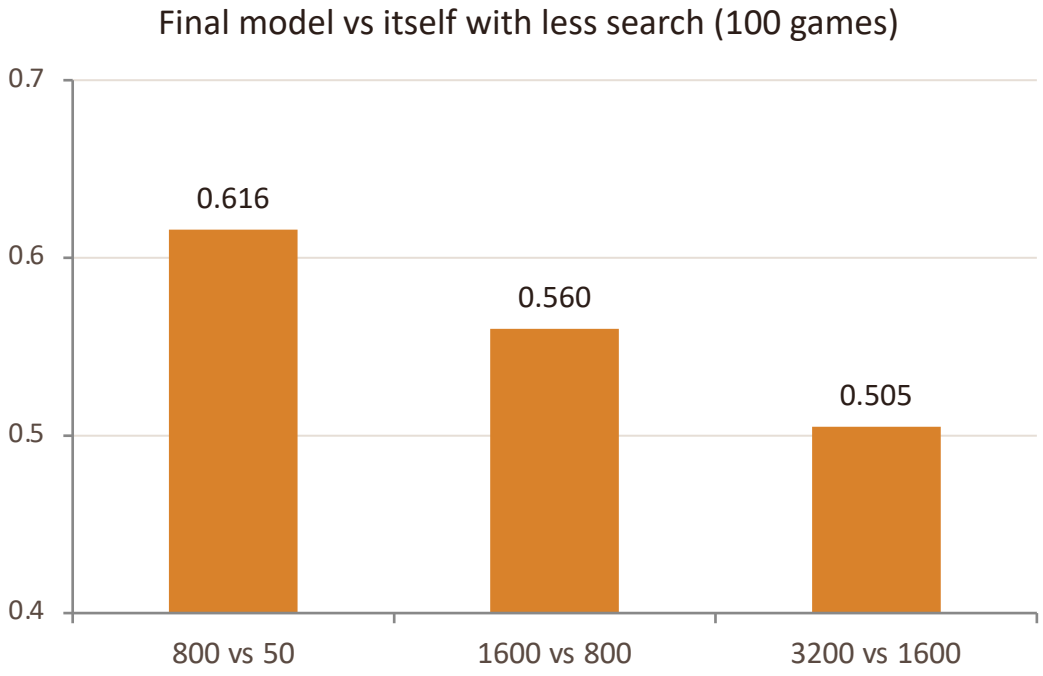
What it learned

- It almost never loses: 5 losses in 400 games
- It checkmates: 23 of 30 games vs greedy, 16 of 30 vs random
- Greedy capture: round 560 drew 99 of 100, round 1280 wins 76

Trying to go further: one idea beat round 560

Idea (all within the rules)	Result	Score vs 560
Bigger network (6×128), twice	Caught up with round 560, never passed it	**0.495**
Counting draws as slightly bad	More decisive games, but riskier and weaker	**0.454**
Even smaller learning steps	No change	**0.515**
Learning from the search's own values	No change	**0.518**
Shorter practice games (300 plies)	2.4× more decisive games per hour, no gain	**0.500**
Averaging 7 trained versions	No change	**0.515**
Resigning hopeless games	Unsafe: the model does not see losses coming	**–**
Gumbel self-play	The one gain: kept improving to round 1220	**0.814**

Thinking longer helps, up to a point



Search steps per move

- The 30–50 limit is for training only
- 800 beat 50 by 23 wins to 0
- 1600 beat 800 by 15 to 3
- 3200 vs 1600: even (7 to 6)
- The browser game uses 1600, about 0.5 s per move

Against people

An experienced player (FIDE blitz 1641) vs round 560

10 games, 800 simulations

2	3	5
AI wins	draws	human wins

AI score 0.350. It kept more material and won one game by checkmate.

The same player vs round 1220

14 games, 800 simulations

0	1	13
AI wins	draw	human wins

AI score 0.036. Far stronger against programs, but not against a person.

Why it does not master the game

	AlphaZero (chess)	ChaturangZero
Self-play games	about 44 million	about 126,000
Search steps per move in training	800	50
Network	about 20 blocks × 256	4 blocks × 64
Hardware	thousands of TPUs	one laptop

350×

fewer games

At our speed, 44 million games would take about 7.5 years of nonstop running on this laptop

Live demo: play the final model

1. `python play_web.py --player Demo --no_log`

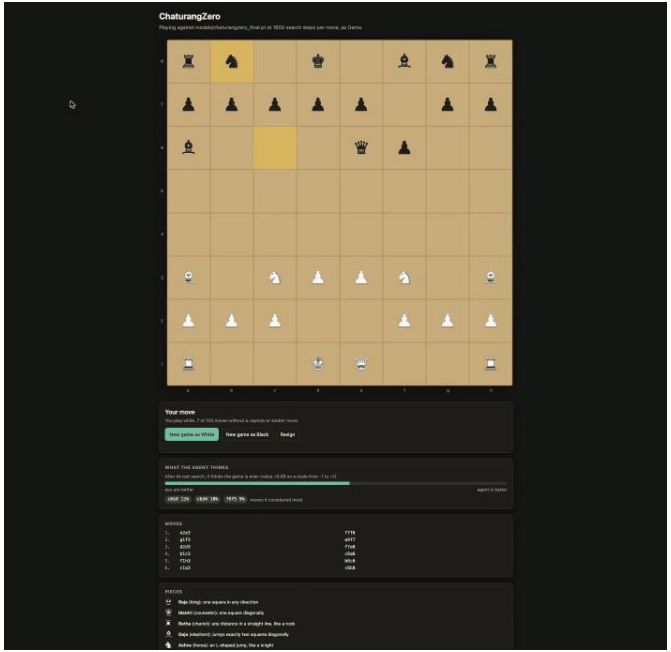
   Opens the board in the browser; no internet needed

2. **Click a piece, then a square**

   Legal moves are highlighted; the AI answers in about half a second

3. **Watch "What the agent thinks"**

   Its evaluation and the three moves it searched most



Backup recording: click to play

What we learned

- The full AlphaZero method works on one laptop, within the rules
- A failed first result, measured and explained, led to the fix: more data
- Gumbel self-play (rounds 1220 and 1280) taught it to finish games; mastery still needs far more games

